Manual Técnico y Guía de Despliegue

Proyecto	FarmaClick - Sistema de gestión para farmacias - Módulo administrativo
Cliente / contraparte	MediFarm
Equipo	D-Software: Alejandro Fuentes Navarrete, Víctor Rubilar Inostroza, Robert González Muñoz
Asignatura	PTY4614 - Capstone
Metodología	Scrum (sprints de 2 semanas)
Código del documento	DOC-09
Versión	1.1
Fecha	21 de octubre de 2026
Repositorio	https://github.com/Alefuentes982/FarmaClick

Control de versiones

Versión	Fecha	Autor	Cambios
1.0	16 de octubre de 2026	Alejandro Fuentes (PO)	Versión inicial, con el sistema desplegado en dev y producción (Incremento 4).
1.1	21 de octubre de 2026	Alejandro Fuentes (PO)	Historia del repositorio en la sección 4 y revisión de redacción.

Contenido

1. Visión general del flujo 3
2. Requisitos 5
3. Desarrollo local con Docker 5
4. Flujo de trabajo con Git 6
5. Base de datos 6
5.1 Migraciones 6
5.2 Crear un proyecto Supabase desde cero 6
6. Oracle Cloud Infrastructure 7
6.1 Red 7
6.2 Llave de despliegue 7
6.3 Máquinas virtuales 7
6.4 Preparar cada VM 8
7. GitHub 8
7.1 Qué hace ci.yml 9
8. Primer despliegue en dev 9
9. Producción con HTTPS 9
9.1 Balanceador 9
9.2 Dominio y certificado 9
9.3 Paso de develop a producción 10
10. Operación (runbooks) 10
11. Solución de problemas 11
12. Costo cero 12
13. Checklist de puesta en marcha 12

1. Visión general del flujo

El código se desarrolla y prueba en el PC con **Docker Compose**. Al hacer ***`git push`*** a la rama **develop**, **GitHub Actions** ejecuta los controles (secretos, pruebas, esquema SQL y compilación), construye las imágenes Docker, las publica en **GitHub Container Registry (GHCR)** y las despliega por SSH en la **VM de dev de Oracle Cloud**. Pasar a producción es fusionar ***develop*** en **main**: el mismo flujo despliega en la **VM de producción**, que atiende detrás de un **balanceador con HTTPS**. La base de datos y la autenticación viven en **Supabase** (un proyecto por entorno) y **el despliegue nunca modifica la base**: las migraciones se aplican a mano.

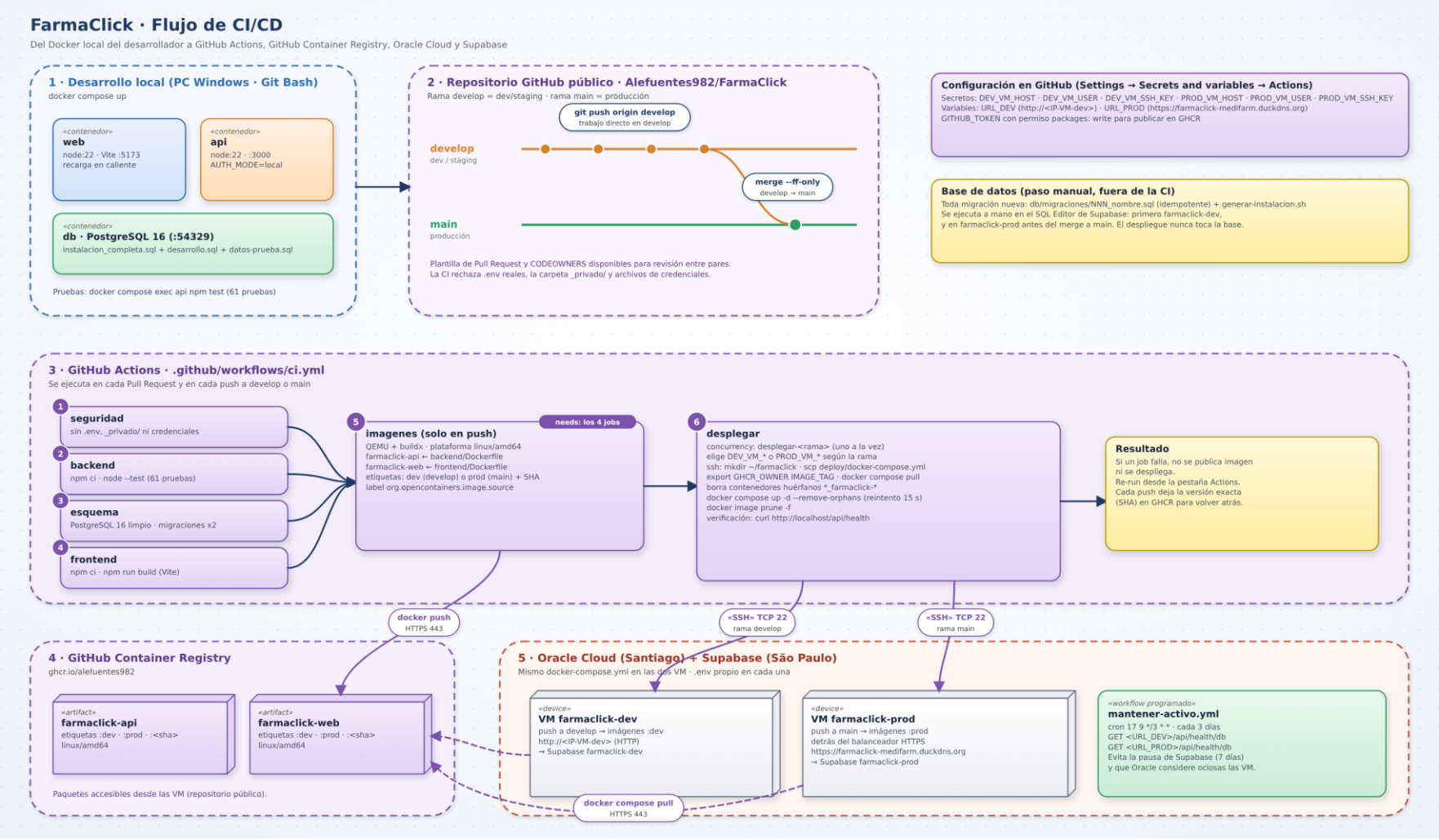


Figura 1. Flujo de CI/CD de punta a punta.

Entorno	Rama	URL	Cómputo	Base de datos
Local	cualquiera	http://localhost:5173	Docker Compose en el PC	PostgreSQL 16 en Docker (:54329)
Dev / staging	develop	http://155.181.171.118	VM farmaclick-dev (E2.1.Micro)	Supabase farmaclick-dev
Producción	main	https://farmaclick-medifarm.duckdns.org	VM farmaclick-prod detrás de lb-farmaclick-prod	Supabase farmaclick-prod

2. Requisitos

Herramienta / cuenta	Para qué	Notas
Git + Git Bash	Control de versiones y consola	Windows: git-scm.com
Docker Desktop (WSL 2)	Entorno local idéntico para todos	Incluye Docker Compose
Cuenta GitHub	Repositorio público, Actions y GHCR	Repositorio Alefuentes982/FarmaClick
Cuenta Oracle Cloud	VM y balanceador Always Free	Región Chile Central (Santiago) sa-santiago-1
Cuenta Supabase	PostgreSQL y Auth gestionados	Plan gratuito: 2 proyectos (dev y prod), región São Paulo
Cuenta DuckDNS	Dominio gratuito de producción	Inicio de sesión con GitHub
Cuenta Gmail con verificación en 2 pasos	Correo saliente (SMTP)	Contraseña de aplicación; clickfarmacias@gmail.com

3. Desarrollo local con Docker

1. Clonar el repositorio y cambiarse a develop:

```
git clone https://github.com/Alefuentes982/FarmaClick.git
cd FarmaClick
git checkout develop
```

1. (Opcional) Para probar correos en local, crear ***backend/.env*** con las variables ***SMTP_**** (ver ***backend/.env.example).*** No hace falta para nada más.
2. Levantar todo. La primera vez tarda unos minutos (descarga imágenes, crea la base e instala dependencias):

```
docker compose up
```

Servicio	Dirección	Detalle
web	http://localhost:5173	Vite con recarga en caliente
api	http://localhost:3000/api/health	Node 22 - AUTH_MODE=local (login de desarrollo)
db	localhost:54329 (postgres / postgres, base farmaclick)	Se inicializa con instalacion_completa.sql, desarrollo.sql y datos-prueba.sql

1. Entrar con ***admin@farmaclick.local*** / ***Admin12345*** o ***vendedor@farmaclick.local*** / ***Vendedor12345***.
2. Ejecutar las pruebas: ***docker compose exec api npm test*** (deben pasar las 61).
3. Reiniciar la base desde cero cuando cambien las migraciones: ***docker compose down -v*** y ***docker compose up.***
4. Si en Windows la API no recarga sola: ***docker compose restart api.***

AUTH_MODE=local solo existe en desarrollo: la API se niega a arrancar con él si **NODE_ENV=production**. En dev y prod el login es con Supabase Auth.

4. Flujo de trabajo con Git

El equipo trabaja **directamente en develop**, con cambios pequeños y frecuentes, y pasa a producción con un **merge fast-forward** de develop a main. Existen una plantilla de Pull Request y CODEOWNERS por si se adoptan ramas ***feature/*.***

```
git checkout develop && git pull
# ... cambios ...
grep -c "<texto nuevo>" <archivo>      # verificar que el archivo se reemplazó
git add -A
git commit -m "Descripción breve del cambio"
git push origin develop                # la rama es develop, no dev
```

El push dispara la CI y el despliegue en dev (sección 8). Revisar en GitHub > **Actions** que el workflow **CI** quede en verde.

Historia del repositorio. Los Sprints 1 a 3 (24-08 al 02-10) se trabajaron en el repositorio previo del equipo, con ramas feature/* hacia dev. El 04-10, antes del Sprint Planning 4, el código se pasó a este repositorio en un commit por módulo, en el orden de sus dependencias: base y login, usuarios, proveedores, productos, órdenes de compra, inventario, pedidos, caja, devoluciones, cuadratura y dashboard. Por eso el historial de git parte el 04-10-2026 y las fechas de esos commits son las de la consolidación, no las de construcción de cada módulo. Para saber cuándo se hizo algo hay que mirar el Sprint Backlog (DOC-04) y el registro de Daily Scrum. Desde el 05-10 el trabajo va directo en develop y el historial tiene las fechas reales.

5. Base de datos

5.1 Migraciones

- Todo cambio de esquema va en una migración **nueva** ***db/migraciones/NNN_nombre.sql***, **idempotente** (se puede ejecutar dos veces) y dentro de **BEGIN; ... COMMIT;.**
- Después se regenera la instalación completa: ***bash db/generar-instalacion.sh*** (actualiza ***db/instalacion_completa.sql).***
- La CI (job esquema) instala todo sobre un PostgreSQL limpio y repite las migraciones para probar la idempotencia.
- Se aplica a mano en el **SQL Editor de Supabase**: primero en **farmaclick-dev**; en **farmaclick-prod** justo **antes** del merge a main.

5.2 Crear un proyecto Supabase desde cero

1. En supabase.com > **New project**: nombre ***farmaclick-dev*** (o ***farmaclick-prod),*** región **South America (São Paulo)**, contraseña de la base guardada en ***_privado/.***
2. **Database > Extensions:** activar ***pg_cron***.
3. **SQL Editor > New query:** pegar el contenido de ***db/instalacion_completa.sql*** y ejecutar (Run). Debe terminar sin errores.
4. Definir la contraseña del rol que usa la API (otra consulta):

```
ALTER ROLE app_backend LOGIN PASSWORD '<clave-larga-sin-simbolos-raros>';
```

1. Programar los jobs y verificarlos:

```
SELECT fn_programar_jobs();
SELECT jobname, schedule, active FROM cron.job;   -- liberar-reservas */5 - baja-vencidos 0 5 -
cierre-automatico 59 *
```

1. (Solo dev) Cargar datos de prueba: ejecutar ***db/datos-prueba/datos-prueba.sql*** (90 productos).
2. **Connect > Transaction pooler (IPv4):** copiar la cadena y armar la ***DATABASE_URL*** con el usuario ***app_backend.<ref>*** y el puerto **6543**:

```
postgresql://app_backend.<ref>:<clave>@aws-1-sa-east-1.pooler.supabase.com:6543/postgres
```

No usar la conexión directa: es IPv6 y desde Oracle Cloud falla con ***ENETUNREACH***. El Transaction pooler no admite sentencias preparadas con nombre; el backend ya está preparado para eso.

1. **Project Settings > API:** copiar ***Project URL, anon key y service_role key*** (la service role es secreta: solo va al ***.env*** de la VM).
2. **Authentication > Sign In / Providers:** desactivar el registro libre (las cuentas las crea el Administrador desde el sistema).

6. Oracle Cloud Infrastructure

6.1 Red

1. Consola OCI, región **sa-santiago-1** > **Networking > Virtual cloud networks > Start VCN Wizard > Create VCN with Internet Connectivity**. Nombre ***vcn-farmaclick,*** CIDR **`10.0.0.0/16`** (crea la subred pública ***`10.0.0.0/24`*** y una privada sin uso).
2. En la **Default Security List** de la subred pública agregar reglas de entrada: TCP **22** desde **`0.0.0.0/0`** (SSH para GitHub Actions) y TCP **443** desde ***`0.0.0.0/0`*** (balanceador). Dejar ICMP.
3. Crear dos **Network Security Groups** en la VCN: ***nsg-dev*** con TCP **80** desde **`0.0.0.0/0`, y *nsg-prod*** con TCP **80** solo desde **`10.0.0.0/16`** (así solo el balanceador llega a producción).

6.2 Llave de despliegue

Crear una llave exclusiva para el despliegue (una sola vez) y guardarla en ***_privado/***:

```
ssh-keygen -t ed25519 -f _privado/farmaclick_deploy -N "" -C "github-actions-farmaclick"
```

6.3 Máquinas virtuales

1. **Compute > Instances > Create instance** dos veces: ***farmaclick-dev y farmaclick-prod.***
2. Imagen **Canonical Ubuntu 24.04**; forma **VM.Standard.E2.1.Micro** (Always Free: 1 OCPU, 1 GB, x86_64).
3. Red: ***vcn-farmaclick***, subred **pública**, asignar IPv4 pública y elegir el NSG (***nsg-dev o nsg-prod***).
4. SSH: pegar el contenido de ***_privado/farmaclick_deploy.pub***. Disco de arranque por defecto.
5. Crear y anotar la IP pública de cada VM (en ***_privado/***).

6.4 Preparar cada VM

```
scp -i _privado/farmaclick_deploy deploy/preparar-vm.sh ubuntu@<IP-VM>:
ssh -i _privado/farmaclick_deploy ubuntu@<IP-VM> "bash preparar-vm.sh"
# preparar-vm.sh instala Docker, abre el puerto 80 en iptables (Ubuntu en OCI lo bloquea) y crea
~/farmaclick
```

Salir y volver a entrar por SSH para que el usuario quede en el grupo docker. Crear el ***.env*** del entorno:

```
ssh -i _privado/farmaclick_deploy ubuntu@<IP-VM>
cd ~/farmaclick && nano .env        # copiar deploy/.env.example y completar
chmod 600 .env
```

Variable	Dev	Prod
DATABASE_URL	pooler del proyecto farmaclick-dev	pooler del proyecto farmaclick-prod
DATABASE_SSL	true (por defecto)	true
SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY	del proyecto dev	del proyecto prod
PORT / NODE_ENV	3000 / production	3000 / production
TRUST_PROXY	1	2 (balanceador + Nginx)
CORS_ORIGIN	(vacío = cualquiera)	https://farmaclick-medifarm.duckdns.org
EMPRESA_NOMBRE / _RUT / _DIRECCION	FarmaClick	FarmaClick (+ RUT y dirección si se quieren en los PDF)
DEVOLUCION_MAX_DIAS	30	30
SMTP_HOST / PORT / SECURE / USER / PASS / FROM	smtp.gmail.com / 587 / false / clickfarmacias@gmail.com / <contraseña de aplicación> / FarmaClick <clickfarmacias@gmail.com>	igual
GHCR_OWNER / IMAGE_TAG	los inyecta la CI	alefuentes982 / prod

Reglas del ***.env:*** sin comentarios al final de la línea, sin comillas (salvo valores con espacios) y con permisos **600**. Después de cambiarlo: ***cd ~/farmaclick && docker compose up -d*** (en dev, antes: ***export GHCR_OWNER=alefuentes982 IMAGE_TAG=dev***).

7. GitHub

1. El repositorio debe ser **público** (Settings > General > Change visibility). Así Actions y GHCR son gratuitos.
2. **Settings > Secrets and variables > Actions > Secrets:** crear ***DEV_VM_HOST*** (IP de dev), ***DEV_VM_USER (ubuntu),*** ***DEV_VM_SSH_KEY*** (contenido completo de ***_privado/farmaclick_deploy***, incluidas las líneas BEGIN/END) y los equivalentes ***PROD_VM_HOST, PROD_VM_USER, PROD_VM_SSH_KEY***.
3. **Variables:** ***URL_DEV = http://155.181.171.118 y URL_PROD = https://farmaclick-medifarm.duckdns.org*** (las usa el workflow mantener-activo).
4. **Packages:** tras el primer build, en el perfil > Packages > ***farmaclick-api*** y ***farmaclick-web*** > Package settings: dejarlos accesibles para que las VM puedan hacer ***docker pull*** sin credenciales (o visibilidad pública).

7.1 Qué hace ci.yml

Job	Se ejecuta en	Qué hace
seguridad	PR y push	Falla si el repositorio contiene ***_privado/***, archivos ***.env*** reales o credenciales.
backend	PR y push	***npm ci --omit=dev*** y ***npm test*** (61 pruebas).
esquema	PR y push	PostgreSQL 16 de servicio: instala ***instalacion_completa.sql*** y repite las migraciones (idempotencia).
frontend	PR y push	***npm ci*** y ***npm run build***.
imagenes	solo push a develop/main, tras los 4 anteriores	QEMU + buildx para linux/amd64; publica ***farmaclick-api*** y ***farmaclick-web*** en GHCR con etiqueta ***dev*** o ***prod*** y el SHA.
desplegar	tras imagenes	concurrency por rama; elige DEV_* o PROD_*; copia el compose por SCP; en la VM: ***docker compose pull***, borra contenedores huérfanos, ***up -d --remove-orphans*** (reintento a los 15 s), ***docker image prune -f***; verifica ***curl http://localhost/api/health***.

mantener-activo.yml se ejecuta cada 3 días (cron `17 9 */3 * *`) o a mano, y consulta ***/api/health/db*** en ***URL_DEV*** y ***URL_PROD*** para que Supabase no pause los proyectos y Oracle no recupere las VM por inactividad.

8. Primer despliegue en dev

1. Con la VM de dev preparada y su ***.env*** listo, hacer un push a develop (cualquier cambio).
2. GitHub > Actions > CI: esperar los seis jobs en verde (unos 6-10 minutos; el build amd64 es lo más lento).
3. Crear el primer administrador del entorno (una sola vez):

```
ssh -i _privado/farmaclick_deploy ubuntu@155.181.171.118
cd ~/farmaclick && docker compose exec api node scripts/crear-admin.js <correo> "<clave>" "Nombre Apellido"
```

1. Abrir **http://155.181.171.118**, entrar con ese usuario y crear sucursales, cajas y usuarios desde el sistema.
2. Verificar: ***curl http://155.181.171.118/api/health/db*** > ***{"ok":true}.***

9. Producción con HTTPS

9.1 Balanceador

1. **Networking > Load balancers > Create load balancer:** nombre ***lb-farmaclick-prod***, público, forma **Flexible 10/10 Mbps** (Always Free), subred pública.
2. **Backend set:** política Weighted round robin; backend ***farmaclick-prod*** puerto **80**; health check HTTP puerto 80, ruta ***/healthz***, código 200.
3. Crear temporalmente un listener HTTP:80 para comprobar que el backend queda **OK**; luego se reemplaza por el HTTPS.
4. Activar los **error logs** (dentro de los 10 GB/mes gratuitos de OCI Logging); dejar los access logs desactivados.

9.2 Dominio y certificado

1. En duckdns.org crear el subdominio ***farmaclick-medifarm*** apuntando a la **IP pública del balanceador**. Guardar el token en ***_privado/.***

2. En la VM de producción, emitir el certificado de Let's Encrypt con validación DNS (no requiere abrir puertos):

```
curl https://get.acme.sh | sh -s email=clickfarmacias@gmail.com
export DuckDNS_Token="<token-duckdns>"
~/.acme.sh/acme.sh --issue --dns dns_duckdns -d farmaclick-medifarm.duckdns.org --server
letsencrypt --keylength 2048
ls ~/.acme.sh/farmaclick-medifarm.duckdns.org/      # .cer  .key  ca.cer  fullchain.cer
```

1. En OCI > ***lb-farmaclick-prod*** > **Certificates and ciphers > Add certificate** (***cert-farmaclick-AAAA-MM***): pegar el ***.cer*** en *SSL certificate*, ***ca.cer*** en *CA certificate* y el ***.key*** en *Private key*.
2. **Listeners > Create listener** `https`: HTTPS, puerto 443, el certificado anterior, cipher suite `oci-default-ssl-cipher-suite-v1`, **sin SSL hacia el backend**. Borrar el listener HTTP temporal.
3. Verificar `https://farmaclick-medifarm.duckdns.org` con candado válido.

9.3 Paso de develop a producción

1. Probar el cambio en dev.
2. Si hay migraciones nuevas, ejecutarlas en el SQL Editor de **farmaclick-prod**.
3. Fusionar y subir:

```
git checkout main && git pull
git merge --ff-only origin/develop
git push origin main
git checkout develop
```

1. Revisar en Actions que **desplegar** quede en verde y probar en https://farmaclick-medifarm.duckdns.org (prueba de humo: ***bash 04_Pruebas/scripts/humo_y_seguridad.sh https://farmaclick-medifarm.duckdns.org***).

10. Operación (runbooks)

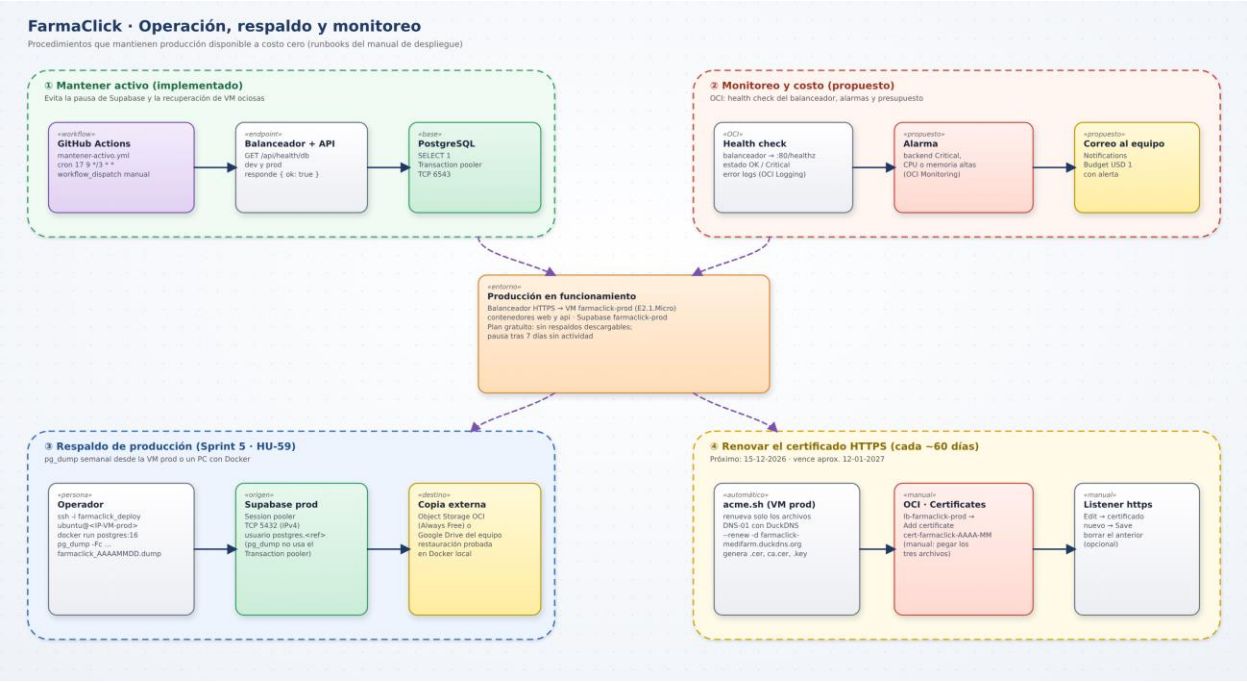


Figura 2. Procedimientos de operación.

Tarea	Comandos / pasos
Entrar a una VM	• ssh -i _privado/farmaclick_deploy ubuntu@<IP-VM> • "Identity file not accessible": la llave no está en esa ruta. "Permission denied (publickey)": es otra llave.
Estado de una VM	• cd ~/farmaclick && docker compose ps • curl -s http://localhost/api/health/db • docker compose logs --tail 50 api
Crear un administrador	docker compose exec api node scripts/crear-admin.js <correo> "<clave>" "Nombre"
Cambiar el .env	cp .env .env.respaldo && nano .env && chmod 600 .env && docker compose up -d && rm .env.respaldo
Aplicar una migración	SQL Editor de Supabase > pegar el archivo > Run; primero dev, después prod; verificar con 3_supabase_diagnostico.sql
Renovar el certificado (cada ~60 días; próximo 15-12-2026)	• acme.sh renueva los archivos solo; si no: ~/.acme.sh/acme.sh --renew -d farmaclick-medifarm.duckdns.org --force • OCI > lb-farmaclick-prod > Add certificate (cert-farmaclick-AAAA-MM) > Listeners > https > elegir el nuevo > Save
Respaldo de producción (HU-59)	• docker run --rm -v "$PWD":/r postgres:16 pg_dump -Fc -f /r/farmaclick_AAAAMMDD.dump "<cadena Session pooler de farmaclick-prod>" • Guardar fuera de Supabase; probar la restauración en local con pg_restore
Volver a una versión anterior	• En la VM: export GHCR_OWNER=alefuentes982 IMAGE_TAG=<etiqueta-anterior> && docker compose pull && docker compose up -d • (cada imagen también se publica con una etiqueta única que identifica la versión)
Kit de diagnóstico	1_oci_cloudshell.sh (Cloud Shell) - 2_vm_diagnostico.sh (VM) - 3_supabase_diagnostico.sql (SQL Editor) - 4_github_diagnostico.sh (Git Bash). Solo leen y ocultan secretos.

11. Solución de problemas

Síntoma	Causa	Solución
Despliegue falla con "Conflict" de nombre de contenedor	Quedó un contenedor <id>_farmaclick-web-1 de un despliegue interrumpido	La CI lo borra antes de up; a mano: docker ps -aq --filter name=_farmaclick- | xargs -r docker rm -f
"removal already in progress"	Dos despliegues simultáneos	concurrency en la CI y reintento tras 15 s
git push origin dev falla	La rama se llama develop	git push origin develop
El PDF sigue con el logo antiguo	El archivo no se actualizó al copiarlo	grep -c LOGO backend/src/core/documentos.js debe dar 5
ENETUNREACH al conectar a Supabase	La conexión directa es IPv6	Transaction pooler IPv4, puerto 6543, usuario app_backend.<ref>
WARN GHCR_OWNER not set (dev)	La variable la inyecta la CI	Inofensivo; exportarla antes de usar docker compose a mano
oci: command not found	Comando de Cloud Shell ejecutado en una VM	Los comandos oci van en OCI Cloud Shell
Backend del balanceador en Critical	Nginx caído o puerto 80 bloqueado	docker compose ps en la VM; NSG nsg-prod; iptables (preparar-vm.sh)
El sitio muestra "no seguro"	Certificado vencido en el balanceador	Runbook de renovación (sección 10)
Supabase pausado	7 días sin actividad	Restaurar desde el panel; revisar que mantener-activo.yml corra

12. Costo cero

- OCI: 2 x VM E2.1.Micro y 1 balanceador flexible de 10 Mbps (Always Free), discos por defecto y Logging dentro de los 10 GB/mes. Pendiente: presupuesto de USD 1 con alerta (Billing > Budgets).
- Si Oracle recupera una VM Always Free por poco uso, convertir la cuenta a Pay As You Go (sigue en USD 0 dentro de los límites).
- Supabase: 2 proyectos gratuitos (el máximo), ~500 MB por base, pausa tras 7 días sin uso.
- GitHub Actions y GHCR: gratis con repositorio público. DuckDNS y Let's Encrypt: gratis.

13. Checklist de puesta en marcha

OK	Paso	Dev	Prod
[ ]	Proyecto Supabase con instalacion_completa.sql, ALTER ROLE app_backend, pg_cron y fn_programar_jobs()	Hecho	Hecho
[ ]	Registro libre desactivado en Supabase Auth		
[ ]	VM creada con NSG correcto y preparar-vm.sh ejecutado	Hecho	Hecho
[ ]	.env completo con permisos 600	Hecho	Hecho
[ ]	Secretos y variables en GitHub (URL_PROD con https)	Hecho	Pendiente URL_PROD
[ ]	Primer despliegue en verde y /api/health/db OK	Hecho	Hecho
[ ]	Administrador creado; sucursales, cajas y usuarios reales	Hecho	Pendiente cajas
[ ]	Balanceador OK, certificado y listener https	-	Hecho
[ ]	Recordatorio de renovación del certificado (15-12-2026)	-	Pendiente
[ ]	Presupuesto de USD 1 en OCI	Pendiente	Pendiente
[ ]	Respaldo con pg_dump y restauración probada	-	Pendiente (HU-59)